Huffman Coding Tree Java
Data Structure and Algorithms

Name: Kevin Ceausu
Student id: B00128171

Introduction

In this project, I have been assigned the task of developing a java application that implements a Huffman Coding system. Huffman is a compression technique that assigns shorter binary codes to more frequent characters and longer binary codes to less frequent characters.
The objective for this project was to build an application based on a text file with each letter of the alphabet having a binary code assigned to it, which will produce a Huffman Tree, helping us encode and decode various messages from a user input through a graphical user interface that is interacting with the system.

System overview

The program is build on 3 main Java classes :

- TreeNode – provided by our lecturer.
- HuffmanTree – builds the Huffman tree, and handles the encoding and decoding logic.
- HuffmanGUI – graphical user interface to allow an easier, more user friendly way of interacting with the application.

The main logic behind the generation of the codes, construction of the tree, encoding and decoding the user input is handled in HuffmanTree.java , while HuffmanGUI.java interacts with HuffmanTree.java as a front end, keeping the logic separated from the user interface.

Class design

1. TreeNode

The TreeNode class is a reference based binary tree node, it contains:

- int freq – the frequency of the characters.
- char symbol – the character store in this node.
- TreeNode left and TreeNode right, referencing to child nodes.

2. HuffmanTree

The HuffmanTree class contains:

- A TreeNode root which is the root of the Huffman tree.
- 2 Array Lists (symbols and codes)

The responsibilities of the class are:

- Get predefined character frequencies from the LetterCountAscending.txt file.
- Build the Huffman tree using the frequencies.
- Generate binary codes for each character.
- Encode a text message into a binary string
- Decode a binary string into text.
- Calculate compression statistics.

3. HuffmanGUI

The HuffmanGUI class extends Jframe and contains:

- A text are for input.
- Buttons for user interaction (show codes, encode, decode, show tree).
- An output zone for encoded/decoded texts.
- An information area for compressions stats, Huffman codes, and the Huffman Tree.

Design Problems:

1. Using the frequency table.

I was not sure how to subtract the data from the table and maintain the correct frequencies.

To solve this problem, I hardcoded the frequency values into the HuffmanTree class in a method called getPredefinedFrequencies(). This method creates Tree Nodes for each letter and its frequency and stores them in a list which acts like a frequency table.

2. Generate binary codes from the tree.

After the Huffman Tree being constructed, next step was to generate binary codes for each character by following the path from the root to each leaf. The codes also needed to be stored to provide fast encoding.

I implemented a recursive method called generateCodesRecursive(TreeNode node, String code). Starting from the root, the method will assign a 0 when recursing to the left child and a 1 when recursing to the right child. When a leaf node is reached, it will store the character in symbols and the binary code in codes.

3. Encoding characters

For encoding, I needed to map each character to its Huffman code.

To solve this problem, I had to reuse the 2 array lists symbols and codes. To encode a message, I go over each character, find its index using symbols.indexOf(ch) and then append the matching Huffman code from codes.

4. Decoding a binary string back into text

Decoding should use the Huffman Tree for this process, the binary string must guide traversal down left and right branches, and character should be recovered when reaching leaf nodes.

To complete this task, I implemented decode(String binary) by walking the tree starting from the root. For each 0 encountered, it would move to the left child, and for each 1, it will move to the right child. When a node with a non null symbol is reached, append that symbol to the results and start all over again from the root until the process ends when there are no more binary strings to decode.

5. Calculation compression ratios using 7 bit ASCII

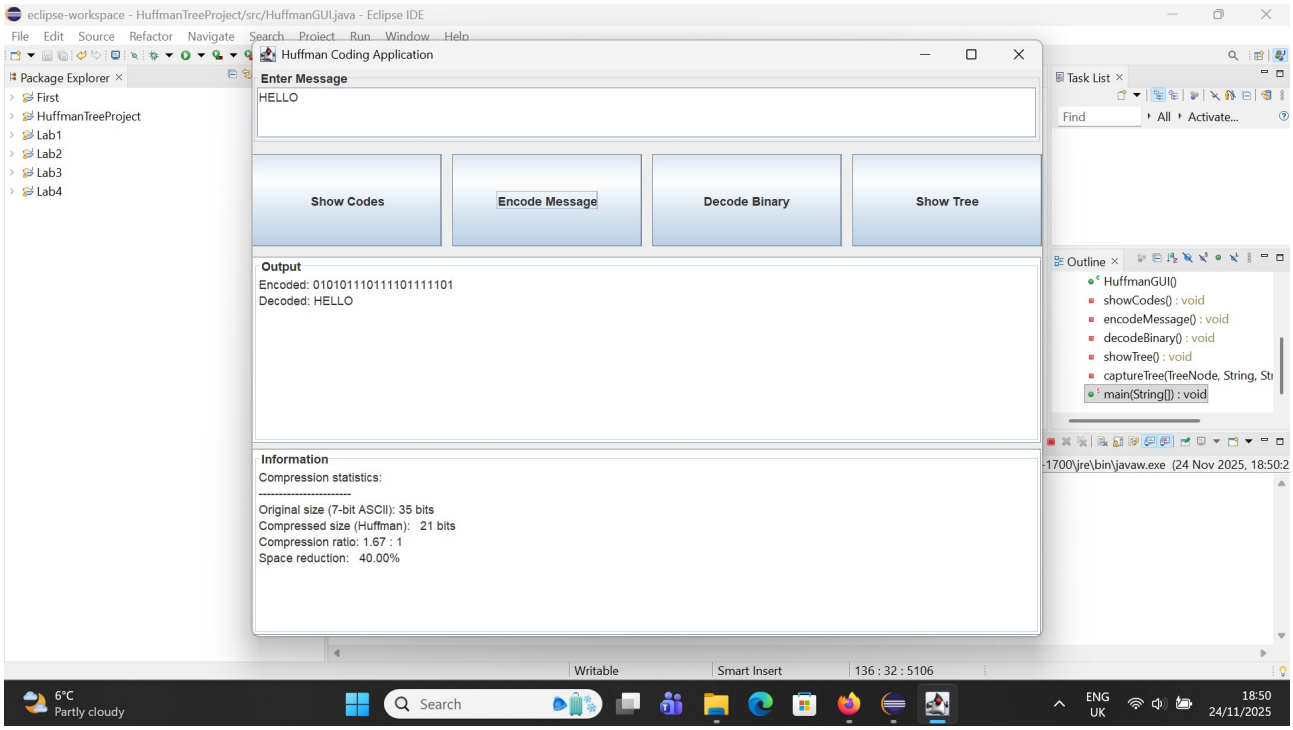
The assignment required to compare Huffman encoded messages to 7-bit ASCII and to display the compression ratio and effectiveness.

To address this, I created getCompressionStats() to count only valid alphabetic characters in the original message. This will count the characters from the original message and multiply the number by 7 to mimic 7 bit ASCII, and compare it against the value we get from counting the encoded message characters. This will return a string to the GUI with all the information and ratios we got from this method.

Testing and Screenshots

In this part I conducted some tests to ensure the application works correctly.

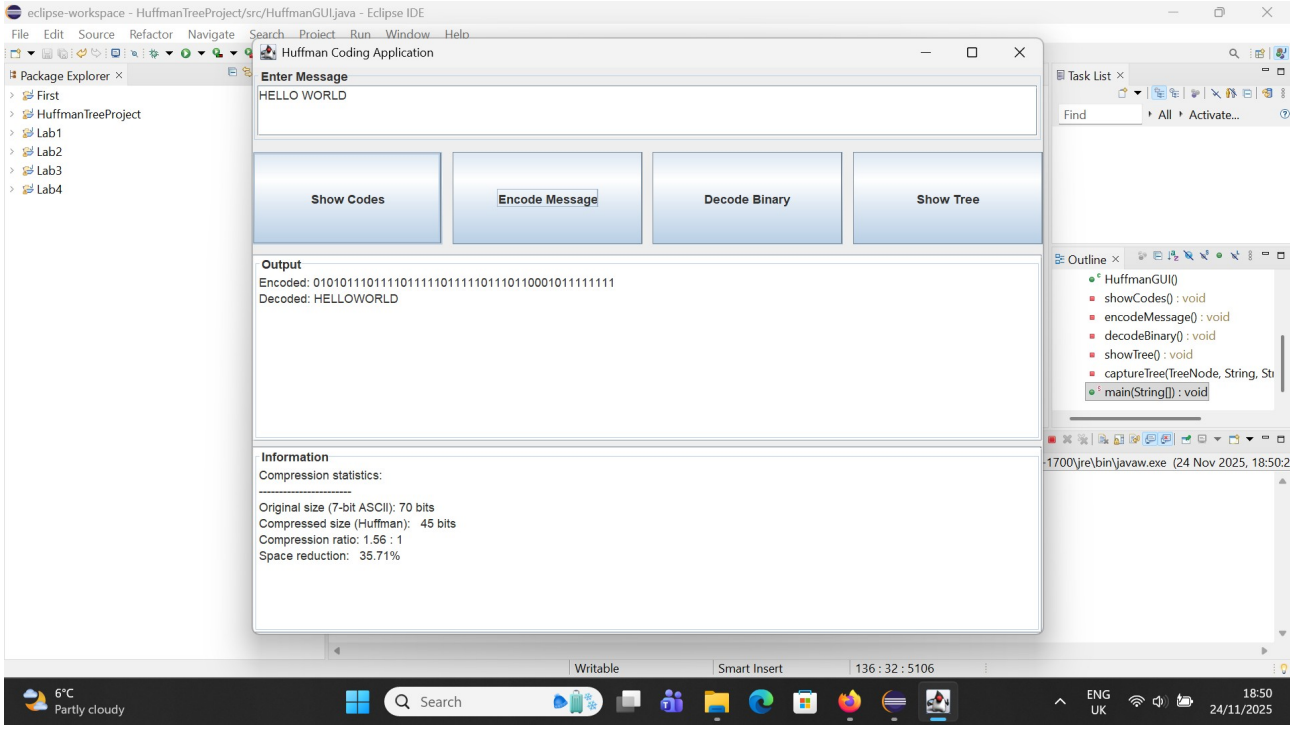
For a starter, I simply wrote HELLO into the user input are in the GUI and this is what I got.



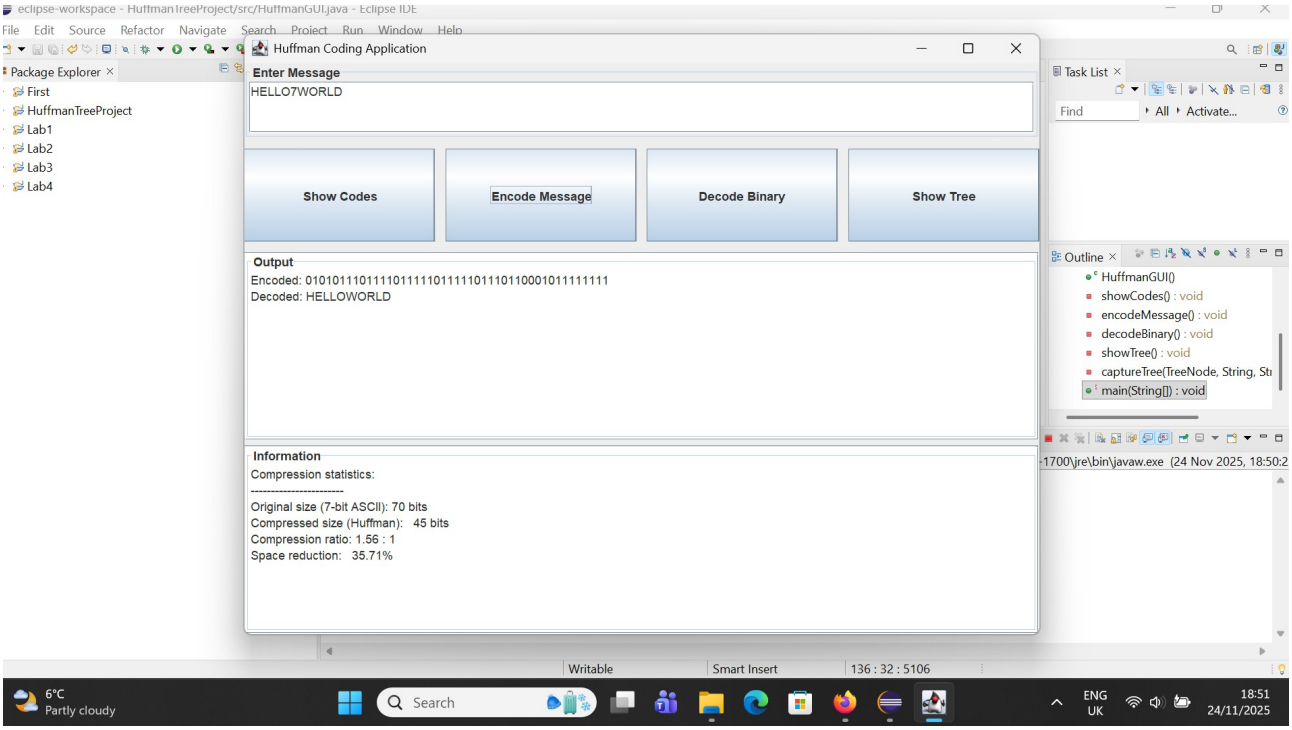
As we can see, it successfully encoded the message, and it also displays all the required information down below, such as original size, compressed size, and the ratios.

In the second test, I proceeded to write HELLO WORLD into the user input are and check if the program will take the spacebar into consideration or will automatically attach the characters together and complete the encoding

As we can see here, it successfully avoided taking the spacebar into consideration.

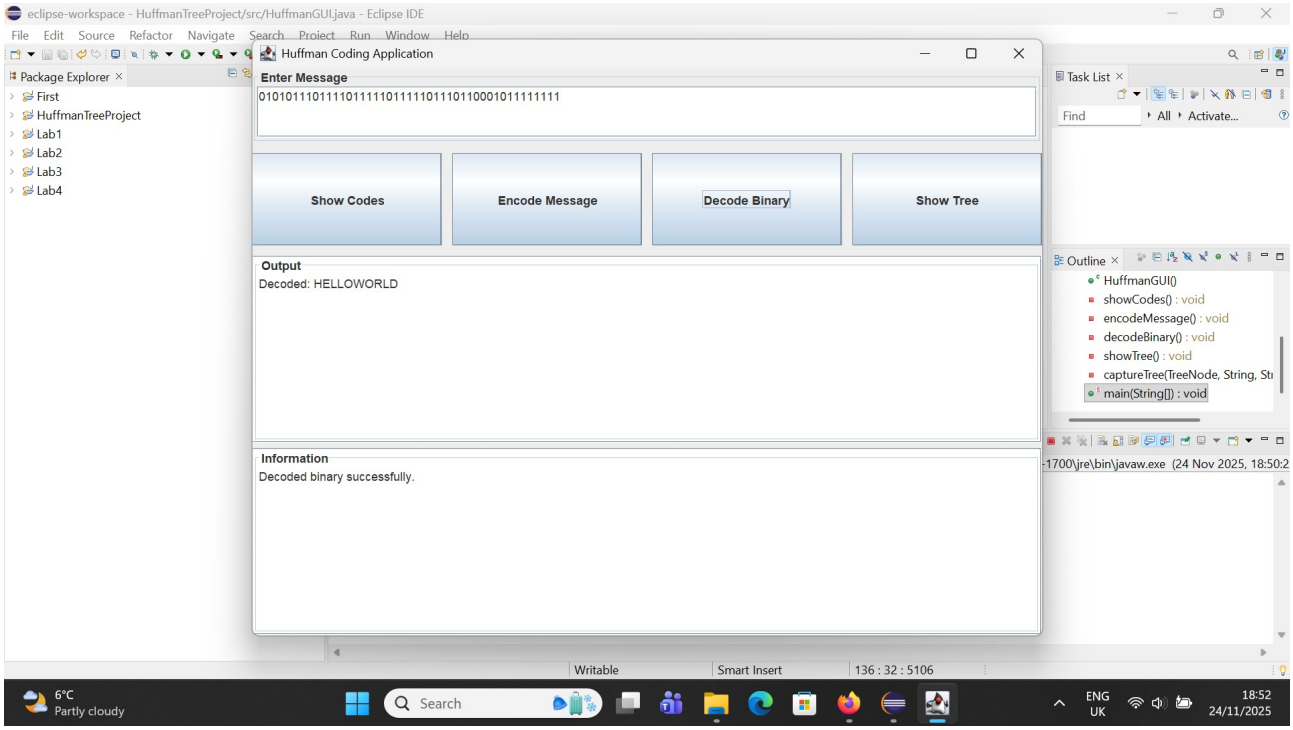


For the next test, I tried to insert a number into the message and see if it manages to avoid numbers and still encode the text successfully.



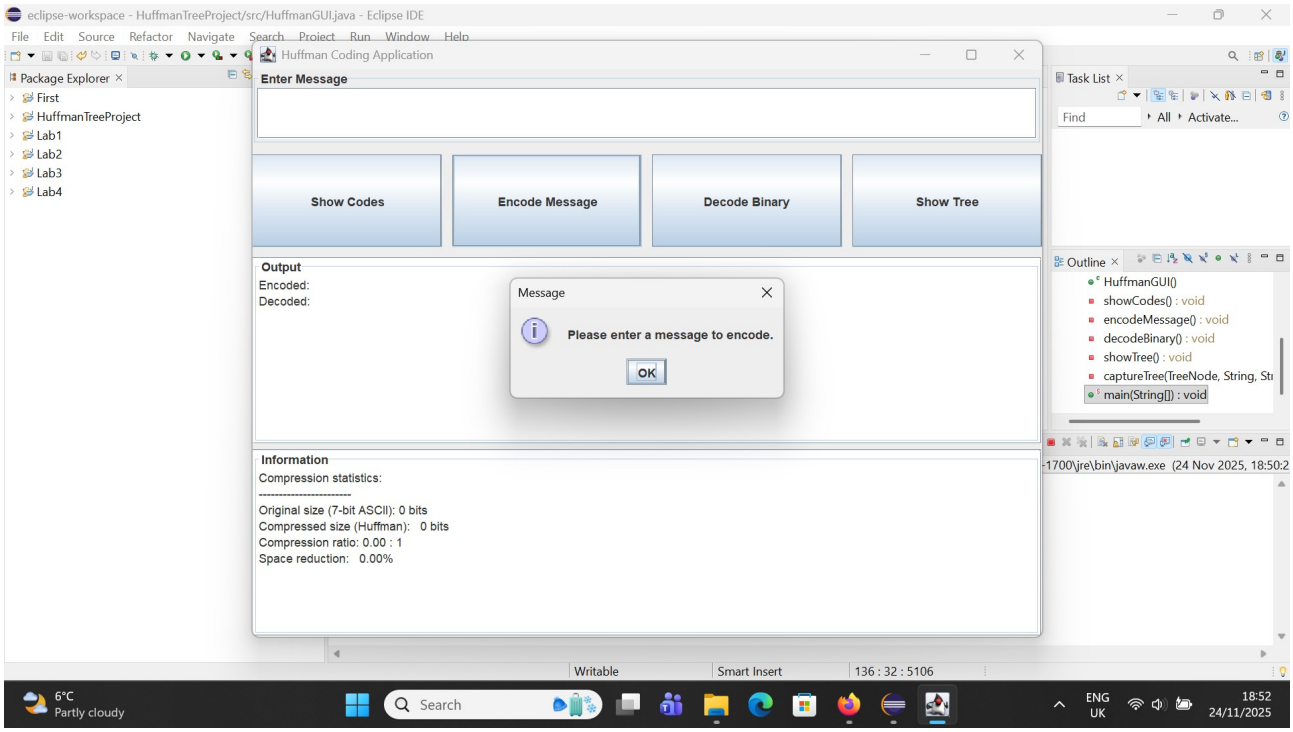
As expected, the application doesn’t take numbers into consideration and moves on with the encoding process.

In the next test, I copied the decoded binary string I got after encoding HELLOWORLD in order to decode it and see the outcome.



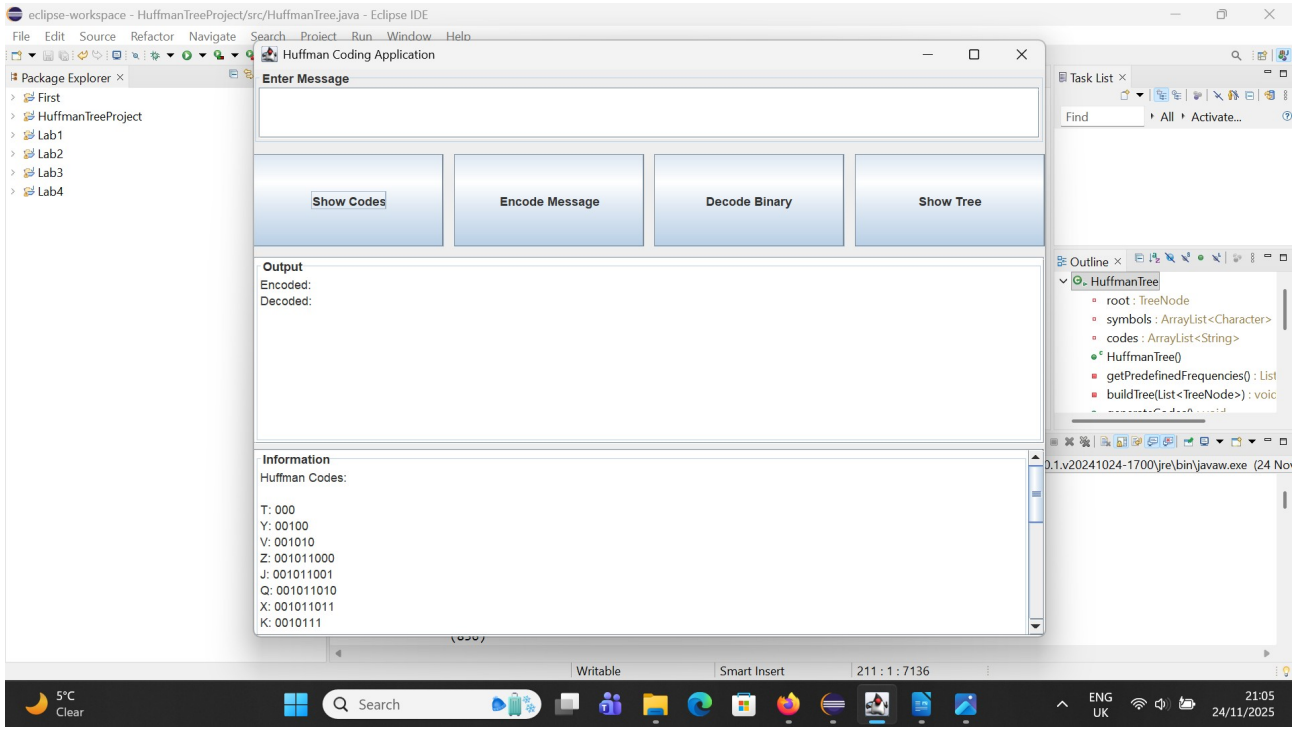
As we can see it, it works correctly and it return HELLOWORLD just as it should.

Next test is seeing what happens if I don’t introduce any user input and just decide to run the encode button with no input inside.



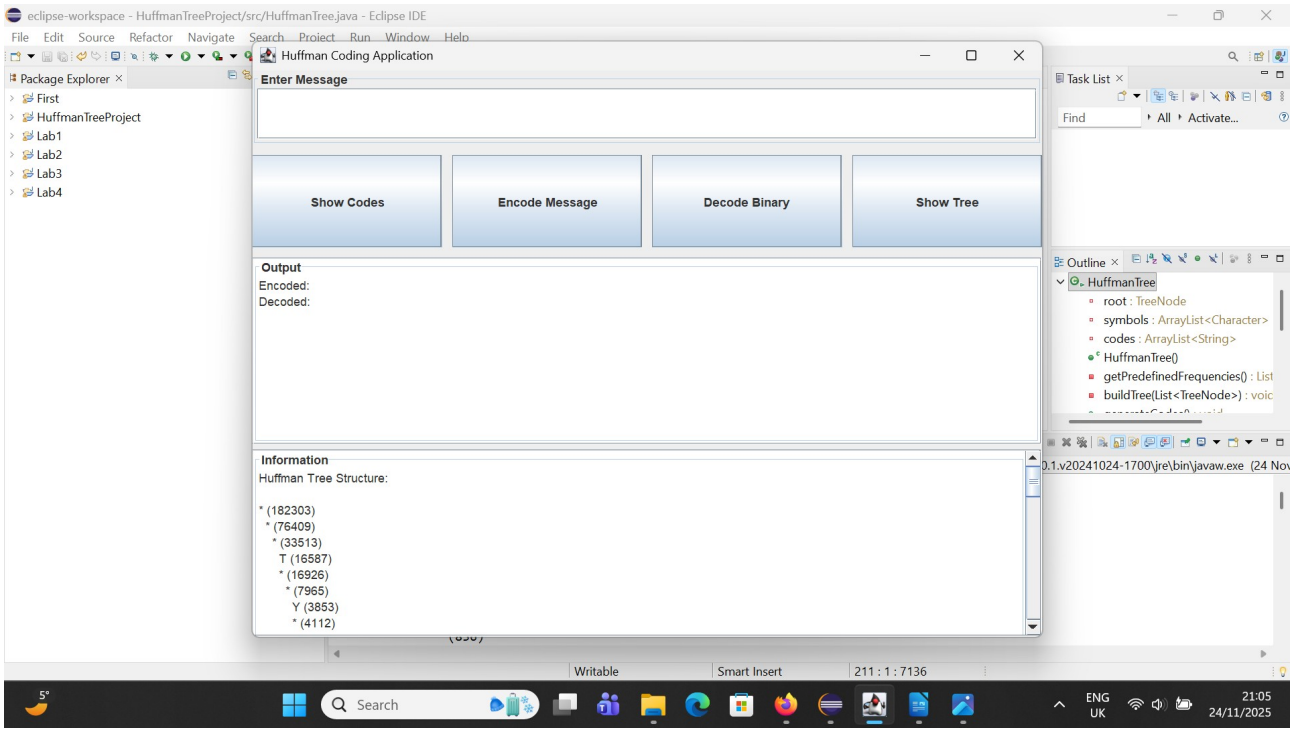
It successfully notices that the user input is empty and pops up a window to ask user to enter a message to encode.

Next image will just be showing what happens when we hit the Show Codes button.



It successfully returns the Huffman Codes.

And for the last test, we will hit the button for Show Tree.



It returns the Huffman Tree structure.

Conclusion

In conclusion, I believe I have successfully created a project that implements a Huffman Coding system in Java that generates a Huffman Tree, encodes a message, decodes a binary string into text, with a simple yet effective program interface.